

BAAR — 블록체인 기반 익명·폐기 가능 인증 프레임워크

Ahmed, Ahmad, Zeshan, Akram (PLOS ONE 21(3):
e0343696, 2026-03-31)

논문 리뷰 **POSTECH** OSTECH
POHANG UNIVERSITY OF SCIENCE AND TECHNOLOGY

오늘의 순서와, 근거를 표시하는 방식

• 순서

- ① 문제 설정과 BAAR가 주장하는 공백 (3~5)
- ② 시스템 모델과 구성요소 (6~8)
- ③ 프로토콜 네 단계 (9~12)
- ④ 이 논문의 핵심 선택: 무엇의 멤버십인가 (13~14)
- ⑤ 보안 주장과 증명 구조 (15~16)
- ⑥ 평가와 수치, 그리고 그 안의 모순 (17~19)
- ⑦ 비판적 검토와 우리 설계와의 대조 (20~26)

• 근거 표시 규칙 — 섞이면 논의가 흐려진다

- "원문": 논문 본문에 있는 내용
- "우리 실측": 이 저장소에서 측정한 값 — 파일명을 함께 적는다
- "발표자 분석": 논문에 없는 해석·비판. 반박 환영

• 시간 배분 — 45분이면 요약을 줄인다

- 9·16·17번은 앞뒤 장과 겹쳐 구두로 넘길 수 있다(약 6분 확보)
- 13~14번(핵심 선택)까지를 25분 지점 통과 목표로, 20~26번(비판·대조)에 최소 25분을 남긴다

문제: 익명성과 폐기는 왜 같이 가기 어려운가

- **블록체인 인증에서 원하는 것 (원문의 문제의식)**
 - 익명성 · unlinkability · 선택적 속성 공개 · 폐기
- **그런데 이 넷은 서로 당긴다**
 - 폐기하려면 "이 크레덴셜이 무엇인가"를 지목해야 하는데, 익명성은 그 지목을 막는다
 - 폐기 상태를 검증자가 조회하면, 그 조회 행위가 "누가 언제 인증 중인가"를 발급자에게 흘린다
 - 폐기된 식별자를 그대로 공개하면 그 사용자를 지목하게 된다(폐기 시 unlinkability 상실)
- **발표자 분석 — 흔한 오해 하나**
 - "공개 폐기 목록은 그 자체로 식별 정보"는 성립하지 않는다 — W3C Bitstring Status List처럼
 - 폐기되지 않은 사용자는 목록에서 식별되지 않는다. 문제는 목록의 존재가 아니라 조회 타이밍과 폐기 대상의 노출이다
- **BAAR의 답: 폐기 상태를 머클 root 하나로 압축해 온체인에 올린다**

기존 접근의 한계 — 논문이 지목하는 것

- **pairing 기반 익명 크레덴셜의 비용 (원문)**

- 기존 프라이버시 보존 스킴 다수가 pairing 기반이라 연산 부담이 크다
- 배포 대상 체인이 secp256k1을 쓰므로 pairing 친화 곡선이 아니다
- 즉 "이론적으로 좋은 스킴"이 배포 대상 위에서 그대로 돌지 않는다

- **각주 — 사실 확인이 필요한 대목**

- 원문은 Bitcoin·Ethereum과 함께 Hyperledger Fabric을 secp256k1 계열로 묶는데,
- Fabric의 기본 MSP는 ECDSA P-256(prime256v1)이다. 동기 문장이라 청중이 짚을 수 있다

- **발표자 분석 — 이 프레이밍의 함의**

- 논문의 설계 제약은 보안이 아니라 배포 가능성에서 온다
- 그래서 SNARK가 아니라 Schnorr Σ -protocol을 고른다 — 23번에서 이 선택의 대가를 본다

BAAR가 주장하는 세 가지 공백 (원문)

	요구	왜 어렵다고 말하는가
(i)	pairing 없이 secp256k1 위에서 동작	기존 익명 크레덴셜 다수가 pairing 전제
(ii)	다속성 선택적 공개 + 가벼운 영지식	SNARK는 무겁고, 단순 서명은 선택적 공개가 안 된다
(iii)	온체인 게시에 적합한 공개·대수시간 폐기	폐기 목록을 통째로 올리면 가스가 감당되지 않는다

논문은 이 셋을 동시에 만족한 선행 연구가 없다고 주장한다.

발표자 분석: (iii)의 "공개(public)"가 핵심이다 — 누구나 검증 가능한 폐기 상태를 목표로 한다.

시스템 모델 — 네 주체 (원문)

주체	하는 일
블록체인	시스템 파라미터·공개키·현재 누산기 root를 게시하는 공개 원장
CIA (Credential Issuing Authority)	초기화, 키 생성, Pedersen 커밋먼트 기반 크레덴셜 발급, 폐기 반영해 root 갱신
검증자 (Verifier)	체인에서 최신 root를 읽어 Schnorr 서명·ZKP·멤버십을 검증
사용자 (User)	크레덴셜 요청, 속성 부분 공개 증명, 폐기 발생 시 witness 갱신

발표자 분석: 검증자가 오프체인 주체라는 점을 기억해 두자. 우리 시스템은 검증자가 스마트컨트랙트다 — 23번에서 이 차이가 결정적으로 작동한다. 그리고 사용자 역할에 "witness 갱신"이 적혀 있다. 그 비용이 평가에는 없다(21번).

구성요소 ①② Pedersen 커밋먼트와 Schnorr ZKPoK (원문)

- **Pedersen 벡터 커밋먼트 — 속성 벡터를 점 하나로**

- $C = H_0^r \cdot \prod_i H_i^{(a_i)}$ (H_0 블라인딩 생성원, H_i 속성별 독립 생성원, r 랜덤)
- perfectly hiding (정보이론적) · ECDLP 하 binding · 속성 수가 늘어도 커밋먼트는 점 하나

- **Schnorr 서명과 지식 증명**

- 서명: $k \rightarrow R = g^k, c = H(R \parallel m), z = k + c \cdot x$. 검증: $R' = g^z \cdot X^{-c}, c = H(R' \parallel m)$
- $\text{PoK}\{ (a_1, \dots, a_n, r) : C = H_0^r \cdot \prod_i H_i^{(a_i)} \}$ (원문 식 (2)), Fiat-Shamir로 비대화형화
- 선택적 공개: 밝힐 속성은 그대로 내고, 감출 속성은 지식만 증명한다

- **발표자 분석 — 여기서 기억할 두 가지**

- 익명성의 힘은 perfect hiding + 매 세션 새 랜덤성에서 온다(계산 가정이 아니다)
- 증명 비용은 생성원 개수, 즉 속성 수 n 에 선형이다 — 19번의 **sublinear** 주장과 충돌한다

구성요소 ③ 머클 동적 누산기 (원문)

- 집합을 **root** 하나로 압축한다
 - 크레덴셜 식별자 C 를 리프로, 누산기 값 = 머클 root R
 - 멤버십 witness $W =$ 인증 경로 $\rightarrow \text{PathHash}(C, W) = R$ 이면 멤버
 - 삽입·삭제는 영향받는 경로의 $O(\log n)$ 노드만 다시 계산한다
- 온체인에 올라가는 것은 **root**뿐 (원문)
 - 개별 커밋먼트는 체인에 저장하지 않는다 — 가스가 낮은 이유이자, 21번 비판의 뿌리다
- 발표자 분석
 - 여기까지는 우리 설계와 같은 그림이다(온체인 상태 32바이트).
 - 갈리는 지점은 딱 하나 — 트리에 무엇을 넣는가. 13~14번에서 본다

프로토콜 ① Setup과 Request (원문)

- **Setup**

- secp256k1 파라미터, Pedersen용 독립 생성원 $\{H_0, \dots, H_n\}$, 빈 폐기 상태로 초기화

- **Request — 사용자가 발급을 요청한다**

- 키쌍 $(x, X=g^x)$ 샘플링, 속성 벡터 a 준비
- 임시 키쌍 (x', X') 생성 — 세션 간 연결을 끊는 장치
- 커밋먼트 C 계산, 지식 증명 PoK_C 생성 $\rightarrow$ CIA에 전송

- **발표자 분석: 사용자가 커밋먼트를 만들어 온다 — CIA는 속성 원문을 보지 않아도 된다**

프로토콜 ② Generation — 여기서 트리에 들어간다 (원문)

- **CIA가 하는 일**

- PoK_C와 X' 아래 서명을 검증한다
- 커밋먼트 C를 누산기에 삽입한다: $R \leftarrow \text{UpdateAccumulator}(R, C)$
- 갱신된 root R'을 온체인에 게시하고, 사용자에게 멤버십 witness W를 돌려준다

- **이 시점의 상태**

- 트리 = 지금까지 발급된 유효한 크레덴셜의 집합. 사용자는 (C, r, a, W)를 든다

- **발표자 분석 — 비용의 씨앗**

- 발급 한 건마다 root가 바뀐다 → 이미 발급된 모든 사용자의 witness가 한 원소씩 낡는다
- 폐기가 드물어도 발급이 잦으면 갱신이 상시로 일어난다 — 21번

프로토콜 ③ 인증 — 검증자가 보는 세 가지 (원문)

#	검사	무엇을 보장하는가
1	$\text{Verify}(X', \sigma)$ — 임시 공개키 아래 서명 검증	이 트랜스크립트를 만든 주체가 비밀키를 안다
2	$\text{Verify}(\text{PoK}_D)$ — 선택적 공개 ZKPoK 검증	감춘 속성을 알고 있고, 밝힌 속성이 커밋먼트와 맞다
3	$\text{VerifyMembership}(R, C, W_C) = 1$	이 크레덴셜이 현재 유효 집합에 있다

세 검사가 모두 통과해야 인증이 성립한다. $\sigma = \text{Sign}(x', (\text{PoK}_D, W))$ 로 증명과 witness를 묶어 보낸다.
발표자 분석: 3번이 폐기를 담당한다 — 폐기 확인이 "현재 root에 대한 멤버십"으로 표현된다.

프로토콜 ④ 폐기 — 리프트를 뺀다 (원문)

- **동작**

- CIA가 크레덴셜 C를 누산기에서 제거하고 새 root R''을 계산해 게시한다
- 이후 C로 만든 증명은 실패한다 — $\text{PathHash}(C, W_{\text{old}}) \neq R''$

- **비용 (원문)**

- $O(\log n)$ 해시 연산, 폐기 가스 약 30K
- 폐기 지연: 뒤의 두 수치(≈ 180 ms / 200-310 ms)가 무엇을 켜 것인지 19번에서 따진다

- **보안 근거 (원문 Lemma 5)**

- 폐기된 크레덴셜에 대해 유효한 witness는 존재하지 않는다 — 만들려면 해시 충돌이 필요하다

- **발표자 분석: 제거 즉시 효력이 생긴다. 유예 창이 없다 — 이 점은 우리보다 낫다(25번)**

핵심: BAAR는 '유효 집합 멤버십'이다

	BAAR (원문)	폐기 집합 방식 (우리 구현)
트리에 담기는 것	발급된 유효 크레덴셜	폐기된 세션·계정 리프
발급 시	커밋먼트를 삽입 → root 변경	아무 일도 없음 (트리 불변)
폐기 시	리프 제거 → root 변경	리프 추가 → root 변경
사용자가 증명하는 것	멤버십 $\text{PathHash}(C,W) = R$	비멤버십 $\text{low} < \text{target} < \text{next}$

원문 확인: accumulator는 "유효 크레덴셜의 root"로 정의되고, 인증은 $\text{VerifyMembership}(R, C, W_C) = 1$ 이다.
다만 원문 안에서 서술이 엇갈린다 — revocation 트리거를 "식별자를 revocation 리스트에 추가"로 적은 대목이 있다. 정의·프로토콜·Lemma 5는 모두 '제거' 쪽과 일관되므로 그쪽을 취한다.
내부 문서 정정 필요: documents/260818_related_work...md의 §3.1 "zk-creds vs BAAR" 표와 §3.3 트리거 항목이 BAAR를 폐기 목록 기반으로 적고 있다.

무엇을 트리에 넣는가 — 방향이 반대다

BAAR: 유효 크레덴셜 집합

발급: 커밋먼트 C 를 삽입 $\rightarrow$ root 변경
폐기: 리프 제거 $\rightarrow$ root 변경
인증: $\text{VerifyMembership}(R, C, W) = 1$
 $\rightarrow$ 트리에 '있어야' 통과

우리: 폐기 집합

발급: 아무 일도 없음 (트리 불변)
폐기: 리프 추가 $\rightarrow$ root 변경
인증: $\text{low} < \text{target} < \text{next}$ (비멤버십)
 $\rightarrow$ 트리에 '없어야' 통과

대가가 비대칭이다. 멤버십은 회로가 단순하다 — 정렬 연결 리스트도, low 리프도, 252비트 정규화와 범위 비교도 필요 없다.

우리 실측: 비멤버십 검증 1개가 6,648 constraints(단독 컴파일). pi_pk_i 는 세션·계정 두 번 쓰므로 13,296 — 전체 19,251의 69%다.

대신 root가 바뀌는 빈도가 다르다: BAAR는 발급+폐기, 우리는 폐기만. 22번에서 이 손익분기를 본다.

보안 주장과 그 근거 (원문)

주장하는 속성	무엇에 기대는가
Unforgeability — 정당한 보유자만 유효 트랜스크립트를 만든다	Schnorr EUF-CMA (A2), ECDLP (A1)
Credential soundness — 발급받지 않고는 인증할 수 없다	머클 누산기 건전성 (A4), 해시 충돌 저항 (A5)
Attribute privacy — 감춘 속성은 보호된다	Pedersen perfect hiding, ZKPoK 영지식성 (A3)
Anonymity / Unlinkability — 세션 간 연결 불가	Pedersen perfect hiding + 매 세션 새 Schnorr 랜덤성
Revocation soundness — 폐기된 크레덴셜은 인증 불가	머클 경로 위조 불가 (A4·A5)
Authority-forgery resistance — 악의적 CIA도 몰래 위조 못 한다	Schnorr 위조 불가 + 해시 충돌 저항

가정: A1 ECDLP · A2 Schnorr EUF-CMA · A3 Fiat-Shamir ZKPoK(ROM) · A4 누산기 건전성 · A5 해시(ROM·충돌 저항).

발표자 분석: 익명성만 정보이론적 요소(perfect hiding)에 기대고 나머지는 계산 가정이다. 그리고 CIA는 발급 자체를 통제하므로 "위조"는 막아도 "부당 발급"은 막지 못한다 — 우리 IdP도 같다.

더 큰 문제: Lemma 4의 게임은 트랜스크립트 구별 불가를 다루지만, 프로토콜은 커밋먼트 c 와 멤버십 witness w 를 검증자에게 평문으로 넘긴다(11번). c 는 크레덴셜당 고정이고 w 는 리프 인덱스를 드러낸다 — 검증자 하나만으로 세션 연결이 자명하고, 두 RP가 담합하면 크로스 서비스 연결까지 된다. 임시키 x 는 공개키만 가릴 뿐이다.

증명 구조 — Lemma 1~7 (원문)

- **게임 기반 논증**

- Unforgeability: 미발급 크레덴셜의 트랜스크립트 출력 → Schnorr EUF-CMA로 환원
- Credential soundness: 누산기에 없는 원소의 witness 생성 → 해시 충돌 필요
- Unlinkability: 두 사용자 중 하나의 트랜스크립트를 구별 → advantage negligible
- Revocation soundness: root 갱신 후 폐기된 크레덴셜로 유효 트랜스크립트 생성 → 경로 위조 필요

- **Lemma 배치**

- 1 위조불가 · 2 크레덴셜 건전성 · 3 속성 프라이버시 · 4 익명성 · 5 폐기 건전성 · 6 권한 위조 저항 · 7 선택적 공개

- **발표자 분석: 증명은 랜덤 오라클 모델에 의존한다(표준 모델 아님)**

평가 셋업 (원문)

- 구현
 - Charm-Crypto (secp256k1), Ethereum + Solidity v0.8.25, 머클 해시 SHA-256
- 측정 환경
 - VMware Ubuntu 22.04.2, 8GB RAM, Intel Core i5-8350U (1.70-1.90 GHz), 각 항목 40회 반복
- 온체인/오프체인 분담 — 뒤의 수치를 읽는 열쇠
 - Schnorr 증명은 오프체인에서 검증한다. 이더리움 트랜잭션 인증은 ECDSA
 - 따라서 가스는 증명 검증 비용이 아니라 상태 게시·조회 비용에 가깝다

성능 ① 시간과 가스 (원문)

항목	BAAR	비교군 [36]	비교군 [37]	가스
Setup	~120 ms	~190	~160	—
Key generation	~150 ms	~230	~210	—
Revocation	~180 ms	~280	~250	~30K (vs [37] ~270K)
Verification	~200 ms	~350	~320	~60K (vs [37] ~100K)
Create	—	—	—	~110K (vs [37] ~135K)

논문은 시간 25~50% 개선, 폐기 가스 약 88% 감소를 주장한다.

발표자 분석: 폐기 가스가 낮은 이유는 온체인에 root 하나만 올리기 때문이다 — 목록을 올리지 않는다. 우리 구조도 온체인 상태는 root 32바이트다.

성능 ② 확장성 주장과, 그 안의 불일치

- 폐기 확장성 (원문)

- 건당 폐기 지연이 규모에 거의 일정하다는 주장 — 200 ms → 310 ms(5,000건 구간)
- 그런데 앞 장 표는 **Revocation**을 ~**180 ms**로 적는다 — 두 값이 무엇을 잴 것인지 논문에 구분이 없다
- 비교군 수치도 "[14] 5,000건에서 약 5초"와 "800-950 ms에서 안정화"가 나란히 나와 대상이 불분명하다

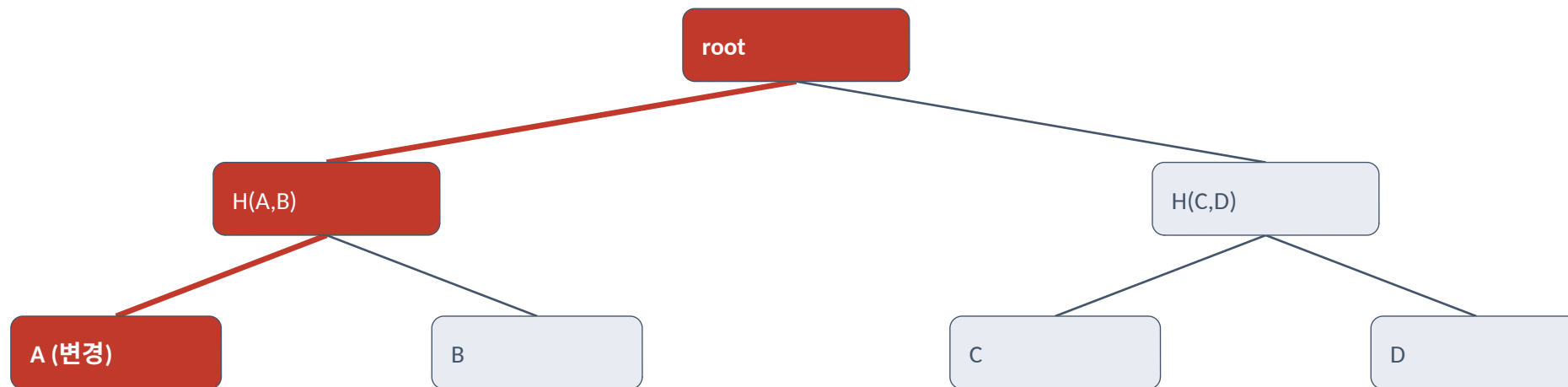
- 속성 확장성 (원문)

- 속성 8~1024개에서 검증 시간 약 15-20 ms로 "sublinear"
- 비교군 [15]은 15초 → 33초

- 발표자 분석 — 여기가 이 논문에서 가장 약한 부분이다

- ① 같은 논문의 Table 4가 인증을 $O(n)$ EC 곱셈으로 적는다. Pedersen/Schnorr PoK는 생성원 n 개에 대한 곱셈이 필수라 n 에 선형이어야 한다
- ② 비교군 [15]의 15초→33초를 "선형 증가"라 부르는데, 속성 128배에 2.2배면 선형이 아니라 강한 sublinear다
- ③ 15초는 SNARK 검증 시간일 수 없다(증명 생성 시간이다). 검증 시간과 나란히 놓으면 부당 비교다

리프 하나가 바뀌면, 모든 사용자의 witness가 한 원소씩 낡는다



붉은 노드 = A가 바뀌면서 값이 달라지는 노드(A의 경로). 이것이 CIA가 $O(\log n)$ 으로 다시 계산하는 부분이다.

B의 witness에는 A가 들어 있다 → 낡는다. C·D의 witness에는 H(A,B)가 들어 있다 → 낡는다.

즉 어떤 리프든 A의 경로와 갈라지는 지점의 형제가 바뀐 노드이므로, 남은 전원의 witness에서 정확히 한 원소가 낡는다.

BAAR에서는 발급도 리프를 추가하므로 발급마다 이 일이 일어난다. 우리(폐기 집합)는 폐기 게시 때만 일어난다 — 종류가 아니라 빈도의 차이다.

단 우리 IMT v2는 삽입 시 low 리프의 next도 갱신하므로 리프가 둘 바뀐다 — 남은 사용자의 witness에서 최대 두 원소가 낡는다.

정직한 대조: 우리 지갑도 같은 IdP에서 목록을 받고(가용성·검열 지점 동일), 증분 조회의 지갑별 커서(since-epoch)는 그 자체로 지문이 된다.

실제 차이는 데이터의 성질이다 — 전원에게 동일한 공개 목록이고 지갑이 root를 스스로 재계산해 대조하므로 누구나 미러링할 수 있고, 잘못된 응답은 즉시 드러난다. 개인별 witness에는 이 성질이 없다.

비판 ① 진짜 문제는 계산량이 아니라 전달 채널이다

- **흔한 비판: "witness 갱신 비용이 평가에 없다" — 이것만으로는 약하다**
 - 우리 실측 1.97 ms는 지압이 전체 리프 배열을 이미 들고 증분만 받는 전제의 값이다 — BAAR 사용자에게 없는 조건이다
 - 로컬 상태가 없는 쪽에 가까운 값은 콜드 재구성이다: $n=32,000$ 에서 3,373 ms(우리 v2). 계산량만으로는 결정적이지 않다
- **버티는 축 — 누가 새 witness를 주는가 (발표자 분석)**
 - 원문은 개별 커밋먼트를 온체인에 저장하지 않고(8번), 오프체인 공개 채널도 명시하지 않는다 → 사용자가 자기 witness를 다시 만들 방법이 없다
 - **원리적 한계는 아니다 — C는 perfectly hiding이라 CIA가 리프 목록을 공개해도 신원이 드러나지 않는다. 다만 논문에 그 설계가 없다**
 - 즉 갱신은 CIA에 의존한다 — 상시 가용성이 필요하고, 검열 지점이 되며,
 - **"누가 언제 갱신을 요청했는가"가 CIA에게 그대로 남는다(익명성 주장과 정면으로 긴장한다)**

비판 ② 손익분기 — 갱신 트래픽 대 회로 비용

- 두 방식은 서로 다른 비용을 낸다. 한 축만 보면 손익분기가 없다 (발표자 분석)
 - root 변경 빈도만 비교하면 $\lambda_{\text{발급}} + \lambda_{\text{폐기}}(\text{BAAR}) \geq \lambda_{\text{폐기}}(\text{우리})$ 라 우리가 항상 이긴다 — 그건 비교가 아니다
- 두 항으로 놓아야 갈림길이 보인다
 - BAAR가 더 내는 것: $(\lambda_{\text{발급}} - \lambda_{\text{재기준화}}) \times N_{\text{users}} \times c_{\text{갱신}} \leftarrow$ 폐기 게시는 양쪽 다 내므로 상쇄된다
 - 우리가 더 내는 것: $\lambda_{\text{인증}} \times \Delta c_{\text{회로}} \leftarrow$ 인증 1회마다 붙는 비멤버십 비용
 - 두 항은 단위가 다르므로 같은 척도(예: 사용자·단위시간당 지갑 CPU ms)로 환산해 비교해야 한다
 - 손익분기는 그렇게 환산한 두 값이 같아지는 지점이다. N이 크고 발급이 잦을수록 BAAR 쪽이 커진다
- 우리 쪽 항의 크기 (우리 실측)
 - 비멤버십 검증 1개 = 6,648 constraints. pi_pk_i는 세션·계정 두 번 인스턴스화한다 $\rightarrow 13,296$
 - 전체 19,251 중 69%가 폐기 확인 비용이다
 - 단 **constraints 69%**가 시간 **69%**는 아니다 — **4,820 $\rightarrow$ 19,251(4.0배)일 때 295.5 $\rightarrow$ 755.8 ms(2.56배)다. 19번에서 BAAR를 때린 논리를 우리도 지켜야 한다**
- 그래서: 우리는 세션마다 발급하므로($\lambda_{\text{발급}} \gg \lambda_{\text{폐기}}$) BAAR 방식이면 갱신이 상시로 흐른다. 반대로 발급이 드문 환경이라면 회로 69%를 아끼는 BAAR 쪽이 낫다

비판 ③ 검증자가 어디 있는가 — 수치를 옮길 수 없는 이유

	BAAR	우리 시스템
검증자	오프체인 주체 (체인에서 root만 읽음)	스마트컨트랙트 (PPIDWallet.execute)
증명 방식	Schnorr Σ -protocol + Fiat-Shamir	Groth16 (circom)
증명이 검증되는 곳	오프체인	온체인 (verifier 컨트랙트)
그래서 생기는 제약	가벼운 증명이면 충분	회로 제약 수가 곧 비용 — Σ -protocol은 온체인 검증에 부적합

발표자 분석: BAAR의 200 ms-60K gas를 우리 시스템에 그대로 옮길 수 없는 근본 이유다.

내부 문서 4.8절은 "Merkle 경로를 회로에 넣으면 9,680 constraints, 500~590 ms"로 추정했으나 실제로는 19,251이다. 빗나간 9,568을 셋으로 나누면: 개수(멤버십 1개 추정 vs 비멤버십 2개 구현) +6,648, 건당 차이(6,648 vs 4,860) +1,788, base 회로 증가(계정 바인딩·pk_i 범위 등) +1,135. 즉 주된 이유는 개수이지 "비멤버십이 훨씬 비싸서"가 아니다.

우리 설계와의 대조 — 같은 도구, 반대 방향

	BAAR (원문)	우리 구현 (우리 실측)
트리 의미 / 증명	유효 크레덴셜 집합 / 멤버십	폐기 집합 / 비멤버십
온체인 상태	root (32B)	root (32B), isCurrentRoot로 최신 하나만 인정
폐기 반영 시점	제거 즉시 (유예 없음)	게시 주기 (prepare → pushRoot → commit)
회로 비용	해당 없음 (오프체인 검증)	pi_pk_i 19,251 constraints (폐기 확인이 69%), 증명 warm 755.8 ms
온체인 실행	검증 ~60K gas (오프체인 증명 전제)	execute() 첫 호출 276,709 gas (Groth16 검증 포함)
트리 갱신 트리거	발급 + 폐기 (둘 다 root 변경)	폐기 게시 + 재기준화 (발급은 트리 불변)
갱신 방식	CIA가 새 witness를 줘야 한다	지갑이 공개 목록으로 스스로 재구성

우리 실측 출처: results/mode2_pi_pk_i_proof_20260904.csv (warm 755.8 ms, 표본 SD ± 15.7 , n=5 / cold 1,650.6 ms), results/mode2_execute_gas_20260904.csv (constraints-gas·비멤버십 개수).

gas 276,709은 첫 execute다 — nonce 0→1 SSTORE(cold)를 포함한다. 같은 지갑의 이후 호출은 그만큼 낮다(미측정).

정직하게: 우리도 root가 바뀌면 모든 사용자의 witness가 낡는다. 빈도가 다를 뿐 성질은 같다.

그리고 우리에게는 재기준화가 있다 — append-only라 슬롯 회수가 그때만 되고, epoch가 오르면 지갑은 증분으로 못 따라잡아 전체를 다시 받는다(한 원소가 낡는 것보다 나쁘다). 드물 뿐이다.

우리가 지는 지점 — 즉시 폐기

- **BAAR:** 리프를 빼는 순간 효력이 생긴다. 유예 창이 없다
- **우리:** 폐기는 게시 주기가 지나야 효력이 생긴다
 - 접수(pendingAdds) → prepare → 온체인 pushRoot → commit 의 2단계 게시를 거친다
 - 그래서 "폐기 지연 = 게시 주기"이고, 주기를 줄이면 가스와 운영 부담이 는다
- **왜 그렇게 했는가 (발표자 분석)**
 - 온체인 게시가 확정되기 전에 IdP가 먼저 전진하면, 지갑이 만드는 witness의 root와 체인의 root가 어긋나 전원이 막힌다
 - 그래서 push가 확정된 뒤에만 IdP가 전진하도록 prepare/commit으로 나뉘었다 — 원자성을 위해 지연을 산 것이다
 - 폐기 하나마다 트랜잭션을 보내면 즉시성은 얻지만 가스와 실패 처리가 사이클마다 붙는다
- **논의거리:** 즉시 폐기를 원한다면 우리는 무엇을 포기해야 하는가?

정리와 논의거리

- 이 논문이 잘한 것

- 배포 제약(secp256k1, pairing 없음)에서 출발해 일관된 설계를 만들었다
- 폐기 상태를 root 하나로 압축해 온체인 비용을 30K gas까지 낮췄다
- 보안 속성을 게임으로 정의하고 Lemma로 나눠 논증했다

- 약한 것

- 속성 확장성 주장이 같은 논문의 복잡도 표($O(n)$)와 충돌하고, 비교 대상이 뒤섞였다(19번)
- witness 갱신의 전달 채널이 설계·평가 어디에도 없다 — CIA 상시 의존이 숨어 있다(21번)
- **unlinkability** 주장과 프로토콜이 어긋난다 — **C**와 **witness**를 평문으로 넘기므로 검증자 하나로 세션 연결이 된다(15번)
- 명시적 limitations·future work 절이 없다

- 논의하고 싶은 것

- λ _발급 vs λ _폐기 — 우리 문제에서 이 비는 얼마인가? 그 값이 멤버십/비멤버십을 가른다(22번)
- 온체인 검증자를 전제하면 Σ -protocol 계열은 선택지에서 빠지는가?
- 우리 내부 문서(§3.1 표·§3.3 트리거)의 BAAR 서술을 정정해야 한다